

Insertion Sort: The One Simple Sort Actually Used

Programming and Data Structures — Week 5, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to implement insertion sort entirely from scratch, understanding its core invariant of a continuously growing sorted prefix; trace insertion sort completely, including its inner shifting loop, on a concrete numeric example; prove precisely that insertion sort runs in $O(n)$ time in the best case and $O(n^2)$ in the worst case; and explain, concretely and specifically, why insertion sort is used as a component inside real production sorting libraries, rather than being purely a teaching exercise.

Outline

Today covers the card-sorting-in-hand analogy, made precise; insertion sort's implementation; a complete trace including the inner shifting loop; the core invariant this algorithm maintains at every step, a continuously growing sorted prefix; a rigorous best-case and worst-case analysis; the concrete, specific reason this algorithm is used inside real production sorting libraries; the MTech-track binary insertion sort variant; an in-class quiz; and a summary.

The card-sorting-in-hand analogy, precisely

This is precisely how most people sort a hand of playing cards in real life: cards already held are kept sorted, and each new card picked up from the table is slid into its correct position among those already in hand, comparing backward only as far as needed. The cards already in hand are never re-examined from scratch — this is exactly the algorithmic idea insertion sort formalizes, and it is worth noticing this is a genuinely different mental model from both of Lecture 1's algorithms.

Insertion sort, in code

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]          # the "new card" being inserted
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]    # shift the larger element right
            j -= 1
        arr[j + 1] = key          # place the key in its correct gap
    return arr
```

The subarray `arr[0:i]` is always fully sorted by the time the outer loop reaches index `i` — this is the core invariant the entire algorithm maintains. `key`, the current element being inserted, is compared backward against this sorted prefix, and every element found larger than `key` is shifted one position to the right, opening a gap. This shifting stops as soon as an element smaller than or equal to `key` is found, or the beginning of the array is reached, at which point `key` is placed into the gap that shifting has opened.

Tracing insertion sort completely on [5, 2, 4, 1, 3]

`i=1: key=2`. Compare to `arr[0]=5`: `5>2`, shift $\rightarrow$ `[5,5,4,1,3]`, `j=-1`. Place 2 at `j+1=0` $\rightarrow$ `[2,5,4,1,3]`
`i=2: key=4`. Compare to `arr[1]=5`: `5>4`, shift $\rightarrow$ `[2,5,5,1,3]`, `j=0`. `arr[0]=2`, not `>4`, stop. Place 4 at `j+1=1` $\rightarrow$ `[2,4,5,1,3]`
`i=3: key=1`. Compare to `arr[2]=5`: shift. `arr[1]=4`: shift. `arr[0]=2`: shift. `j=-1`. Place 1 at `j+1=0` $\rightarrow$ `[1,2,4,5,3]`
`i=4: key=3`. Compare to `arr[3]=5`: shift $\rightarrow$ `[1,2,4,5,5]`. `arr[2]=4`: shift $\rightarrow$ `[1,2,4,4,5]`. `arr[1]=2`: not `>4`, stop. Place 3 at `j+1=2` $\rightarrow$ `[1,2,3,4,5]`

Tracing the full run on `[5, 2, 4, 1, 3]`: at `i=1`, key 2 shifts past 5, landing at index 0, giving `[2, 5, 4, 1, 3]`. At `i=2`, key 4 shifts past 5 only, stopping at 2, giving `[2, 4, 5, 1, 3]`. At `i=3`, key 1 shifts past all three of 5, 4, and 2 in turn, landing at index 0, giving `[1, 2, 4, 5, 3]`. At `i=4`, key 3 shifts past 5 and 4, stopping at 2, giving the fully sorted `[1, 2, 3, 4, 5]`. Notice that at every single step, the portion of the array before index `i` was already fully sorted — the invariant held throughout, not just at the end.

A second complete trace — nearly sorted input

Tracing the nearly-sorted input `[1, 2, 3, 5, 4]`, where only the last two elements are out of place, shows the inner `while` loop body executing zero times for `i=1, 2`, and `3` — each of those keys is already

in its correct position relative to the sorted prefix, requiring only a single comparison to confirm. Only at $i=4$ does any real shifting occur, moving 5 one position to the right before placing 4 correctly. This is direct, concrete evidence of insertion sort's strong performance on nearly-sorted data, previewing the quiz at the end of this lecture.

The invariant, stated formally

The invariant “before processing index i , $\text{arr}[0:i]$ is fully sorted” can be verified by induction, the same reasoning tool used in mathematical proofs. The base case, $\text{arr}[0:1]$, a single element, is trivially sorted — there is nothing to compare. The inductive step assumes $\text{arr}[0:i]$ is already sorted and shows that correctly inserting $\text{key} = \text{arr}[i]$ into its proper position produces a sorted $\text{arr}[0:i+1]$. Since both the base case and the inductive step hold, the invariant holds for every i , which is precisely what proves the algorithm produces a fully sorted array once i reaches the end.

Best case: an already-sorted array

On an already-sorted input, the condition $\text{arr}[j] > \text{key}$ is false on the very first check for every value of i , since each key is already greater than or equal to everything before it — the inner `while` loop body never executes at all. Total work reduces to exactly one comparison per outer-loop iteration, giving $n - 1$ comparisons total, genuinely $O(n)$. This matches bubble sort's best-case behavior from Lecture 1, for the same underlying reason: when nothing in the input is actually out of order, no shifting or swapping work is needed at all.

Worst case: a reverse-sorted array

On a strictly decreasing input, every new key is smaller than everything already placed, forcing the inner loop to shift past all i elements already in the sorted prefix before finding its place at the very front. Summed across all outer iterations, this gives $1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2}$ total shifts, exactly $O(n^2)$ — the identical closed-form result derived for selection sort in Lecture 1, though arrived at through a structurally different mechanism (shifting here, versus scanning for a minimum there).

Cost as a function of “how far out of place,” precisely

This behavior can be stated with complete precision using the concept of an inversion: a pair of positions (i, j) with $i < j$ but $\text{arr}[i] > \text{arr}[j]$ — a pair currently in the wrong relative order. Insertion sort's total number of element shifts across the entire sort equals exactly the number of inversions present in the original input, no more and no less. An already-sorted array has zero inversions, giving zero shifts. A reverse-sorted array has every possible pair inverted, giving $\frac{n(n-1)}{2}$ inversions and exactly that many

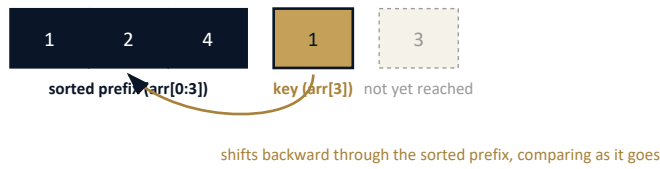
shifts. This is a sharper, exact characterization of insertion sort's cost than “best case” and “worst case” alone provide — the actual cost is a precise function of how disordered the specific input is.

Why this one is actually used: hybrid sorting

Python's built-in sort (Timsort, used by both `sorted()` and `list.sort()`) explicitly switches to insertion sort once a subarray shrinks below roughly 64 elements, even though the overall algorithm is a sophisticated merge-based approach covered in Week 6. The reason is concrete: insertion sort's very low overhead per element beats the recursive setup and merging cost of an algorithm like merge sort once n is small enough that $O(n^2)$ versus $O(n \log n)$ barely differs in absolute terms, but insertion sort's simplicity wins on the actual constant factors involved. This is the central lesson of this lecture: real production sorting libraries are hybrids, deliberately using different algorithms for different input sizes, rather than committing to one algorithm unconditionally.

The picture: the growing sorted prefix

State partway through: $i = 3$



The array is always split into a sorted left prefix and an unsorted right portion; the next unsorted element is slid into its correct place in the prefix.

This is the single image worth retaining from this lecture: at every step, the array is split into a sorted left prefix and an unsorted right portion, and the algorithm's entire job is repeatedly taking the first element of the unsorted portion and sliding it into its correct place within the already-sorted prefix.

MTech addition — binary insertion sort

```
import bisect

def binary_insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        pos = bisect.bisect_left(arr, key, 0, i)    # O(log i) search
        arr[pos+1:i+1] = arr[pos:i]                # shift block right
```

```
arr[pos] = key
return arr
```

A natural-seeming optimization uses binary search, from Week 1 Lecture 4, to find the correct insertion position in $O(\log i)$ comparisons instead of a linear backward scan. This genuinely reduces the total number of comparisons across the whole sort to $O(n \log n)$. However, the total running time remains $O(n^2)$, because the elements between the found position and index i must still be physically shifted one at a time — an $O(n)$ operation per insertion, unaffected by how quickly the position itself was located. This is an important and often-missed lesson: identifying the true bottleneck of an algorithm (here, shifting, not comparing) is essential before optimizing the wrong part.

Counting inversions in [1, 2, 3, 5, 4]

Checking every pair in [1, 2, 3, 5, 4]: the only pair in the wrong relative order is (5, 4), giving exactly 1 inversion. The complete trace earlier in this lecture found exactly 1 shift occurred during the entire sort. This match is not a coincidence — it is the precise relationship between inversions and shift count this lecture established, and it holds for every input, not just this particular example.

Exercise

As an exercise, trace `insertion_sort` by hand on the already-sorted input [1, 2, 3, 4, 5], counting the exact number of comparisons made, and confirm it matches the $O(n)$ best-case claim precisely — it should be exactly $n - 1 = 4$ comparisons. Separately, implement `binary_insertion_sort` and confirm empirically, by instrumenting both versions to count operations, that it makes fewer comparisons but a similar number of element moves compared to plain insertion sort on a reverse-sorted input — directly confirming this lecture's claim about where the true bottleneck lies.

Comparing all three simple sorts, side by side

Placing all three simple sorting algorithms from this week side by side: bubble sort and insertion sort both achieve $O(n)$ on already-sorted input, while selection sort structurally cannot; all three degrade to $O(n^2)$ in the worst case; their shift or swap counts differ meaningfully — bubble sort's varies up to the full inversion count, selection sort's is always exactly $n - 1$, and insertion sort's equals the inversion count exactly, a precise result unique to this lecture; and bubble sort and insertion sort are both stable, while selection sort as implemented is not. No single row crowns one algorithm as universally best — the right choice, again, depends on the actual workload.

Quiz — best case or worst case?

For each of these three inputs, decide whether insertion sort's performance is closer to its best case or its worst case, before checking the answer: [1, 2, 3, 4, 5, 6]; [6, 5, 4, 3, 2, 1]; and [1, 2, 3, 4, 6, 5], where only the last two elements are out of order.

Where insertion sort's limits show up

It is worth being honest about where insertion sort's limits actually show up. A large array in a genuinely random order has, on average, approximately $\frac{n(n-1)}{4}$ inversions — roughly half of the maximum possible — which means insertion sort's *typical* running time on large random data is still $O(n^2)$, not merely its rare worst case. This is not a contrived pathological scenario; it is the ordinary expected behavior on unstructured data at scale, and it is precisely the gap that Week 6's merge sort and quicksort exist to close, both guaranteeing (or, for quicksort, expecting) $O(n \log n)$ regardless of how the input is initially ordered.

Quiz — answer

[1, 2, 3, 4, 5, 6] is already sorted, giving insertion sort's best case, $O(n)$. [6, 5, 4, 3, 2, 1] is fully reversed, giving its worst case, $O(n^2)$, with maximal shifting at every single insertion. [1, 2, 3, 4, 6, 5] is nearly sorted, with only one adjacent pair out of place — this is close to the best case in practice, since every insertion except the very last requires zero shifting, and even the last requires only one shift. This third case illustrates a genuinely important practical property: insertion sort performs very well specifically on *nearly*-sorted data, not only on perfectly sorted data — a property later sorting algorithms in Week 6 do not all share equally.

Summary

Insertion sort maintains a continuously growing sorted prefix, inserting each new element into its correct position by shifting larger elements rightward. Its best case is $O(n)$, on already-sorted or nearly-sorted data; its worst case is $O(n^2)$, on reverse-sorted data — with the same exact closed-form comparison count as Lecture 1's selection sort. Real production sorting libraries such as Timsort genuinely use insertion sort internally for small subarrays, a specific and concrete use case, not merely a historical curiosity. Binary insertion sort reduces the comparison count to $O(n \log n)$ but leaves total running time at $O(n^2)$, since shifting elements, not comparing them, is the actual bottleneck — an important lesson in correctly identifying what an algorithm's true cost is dominated by before attempting to optimize it. Next lecture moves past the $O(n^2)$ barrier entirely with merge sort and quicksort.